UNIT IV – MULTIMODAL GENERATIVE AI APPLICATIONS

QUESTION 5 – SPEECH-TO-TEXT APPLICATION

Engineering Voice Query to Written Text

Practical Implementation Report

Question	Develop a Speech-to-Text application that converts a user's spoken engineering-related query into written text using a pre-trained AI model.
Folder Name	Q5_Speech_To_Text
Python File	speech_to_text.py
AI Model / Service	Whisper through Hugging Face Inference API

1. AIM

To develop a Speech-to-Text application that accepts a spoken engineering-related query and converts it into readable written text using a pre-trained AI speech-recognition model.

2. OBJECTIVES

- Accept an engineering-related audio query through a web interface.
- Send the uploaded speech to a pre-trained speech-recognition model.
- Convert the spoken query into written text.
- Display the recognized engineering question clearly.
- Demonstrate the practical use of speech AI in engineering education.

3. SOFTWARE AND HARDWARE REQUIREMENTS

- Python 3.x
- Streamlit
- Hugging Face Hub / Inference API
- Pre-trained Whisper speech-recognition model
- Internet connection for API inference
- Computer or laptop with microphone/audio support

4. FOLDER AND FILE STRUCTURE

```
UNIT_IV_Multimodal_Generative_AI
└── Q5_Speech_To_Text
    └── speech_to_text.py
```

5. TECHNOLOGIES USED

Technology	Purpose	Role
Python	Programming	Application implementation
Streamlit	Web Interface	Audio upload and output display
Hugging Face Inference API	AI Inference	Connects the app to the speech model
Whisper	Speech Recognition	Converts spoken audio into text

6. SYSTEM WORKFLOW

Spoken Engineering Query → Audio Upload → Streamlit Application → Hugging Face Inference API → Whisper Speech Model → Recognized Text

Figure 3. Speech-to-Text workflow.

7. STEP-BY-STEP IMPLEMENTATION

Step 1: Create the Q5_Speech_To_Text folder inside UNIT_IV_Multimodal_Generative_AI.

Step 2: Create speech_to_text.py inside the Q5 folder.

Step 3: Install Streamlit and huggingface_hub.

Step 4: Create a Hugging Face access token and store it securely as HF_TOKEN.

Step 5: Initialize the Hugging Face InferenceClient using the token.

Step 6: Create an audio uploader that accepts WAV, MP3, FLAC, M4A and OGG files.

Step 7: Upload a spoken engineering query.

Step 8: Send the audio file to the pre-trained Whisper speech-recognition model.

Step 9: Receive the recognized text from the API.

Step 10: Display the audio player and recognized engineering query.

Step 11: Test the application with multiple engineering-related voice queries.

8. IMPLEMENTATION CODE

```
import streamlit as st
from huggingface_hub import InferenceClient
import os
import tempfile

st.set_page_config(
    page_title="Engineering Speech-to-Text",
    page_icon="🎤",
    layout="centered"
)

st.title("🎤 Engineering Speech-to-Text Application")

st.write(
    "Upload a spoken engineering-related query and convert "
    "it into written text using a pre-trained AI speech-recognition model."
)

HF_TOKEN = os.getenv("HF_TOKEN")

if not HF_TOKEN:
    st.error("HF_TOKEN is not configured.")
    st.stop()

client = InferenceClient(api_key=HF_TOKEN)

audio_file = st.file_uploader(
    "Upload an audio file",
    type=["wav", "mp3", "flac", "m4a", "ogg"]
)

if st.button("📝 Convert Speech to Text"):

    if audio_file is None:
        st.warning("Please upload an audio file first.")

    else:
```

```
try:
    with tempfile.NamedTemporaryFile(
        delete=False,
        suffix=os.path.splitext(audio_file.name)[1]
    ) as temp_file:
        temp_file.write(audio_file.getbuffer())
        temp_path = temp_file.name

    st.audio(audio_file, format=audio_file.type)

    with st.spinner("Converting speech into text..."):
        result = client.automatic_speech_recognition(
            audio=temp_path,
            model="openai/whisper-large-v3"
        )

    st.subheader(" Recognized Engineering Query")
    st.success(result.text)

    os.remove(temp_path)

except Exception as e:
    st.error("Speech recognition failed.")
    st.write(str(e))
```

9. EXECUTION COMMAND

```
streamlit run ".\Q5_Speech_To_Text\speech_to_text.py"
```

10. TEST INPUTS

- Explain Ohm's Law.
- What is database normalization?
- What is virtual memory in operating systems?

11. OBSERVED OUTPUT

The uploaded audio containing the spoken engineering query was successfully converted into written text. The screenshots below show the actual implementation output obtained during testing.

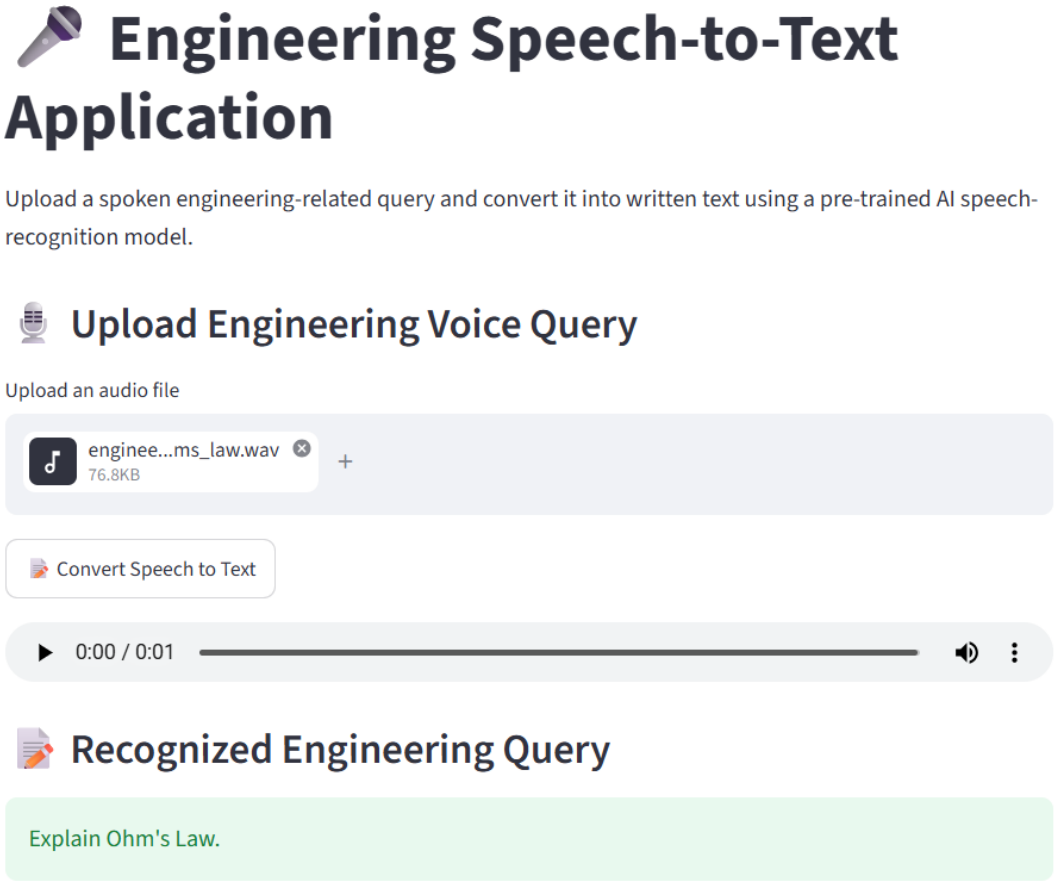

Figure 1. Speech-to-Text application successfully transcribing the spoken Ohm's Law query.

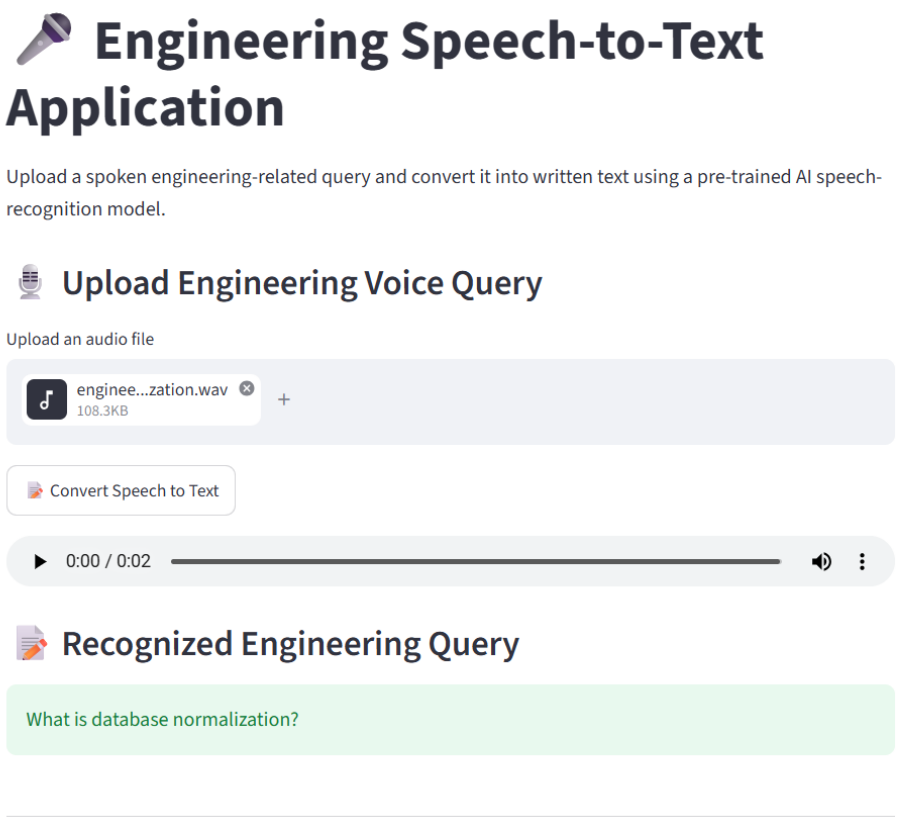

Figure 2. Speech-to-Text application successfully transcribing the spoken database-normalization query.

12. TEST RESULTS

Test Case	Spoken Query	Recognized Text
1	Explain Ohm's Law.	Explain Ohm's Law.
2	What is database normalization?	What is database normalization?
3	What is virtual memory in operating systems?	What is virtual memory in operating systems?

13. RESULT

The Engineering Speech-to-Text application was successfully developed. It accepts spoken engineering-related queries as audio input and converts them into written text using a pre-trained Whisper speech-recognition model through the Hugging Face inference service. The actual screenshots demonstrate successful transcription of engineering questions.

14. ADVANTAGES

- Converts natural spoken queries into text automatically.
- Supports engineering-related voice interaction.
- Uses a pre-trained AI speech model, so model training is not required.
- Provides a simple Streamlit interface.
- Can be extended to multilingual speech recognition.

15. LIMITATIONS

- Recognition quality depends on audio clarity and background noise.
- The application requires internet access for the inference service.
- Very technical terminology may occasionally be transcribed incorrectly.
- API availability and usage limits can affect response time.

16. FUTURE ENHANCEMENTS

- Add direct microphone recording inside the web application.
- Support more Indian languages for engineering queries.
- Add voice activity detection and noise reduction.
- Connect the recognized query to an engineering chatbot for automatic answers.
- Store transcriptions for later analysis.

17. CONCLUSION

The developed Speech-to-Text application demonstrates a practical multimodal Generative AI workflow for engineering education. Spoken engineering queries can be converted into written text automatically using a pre-trained speech-recognition model. The successful test results confirm that the application satisfies the required Speech-to-Text implementation.

18. VIVA / KEY POINTS

- Speech-to-Text converts human speech into written language.
- Whisper is a pre-trained AI speech-recognition model.
- Hugging Face Inference API provides access to the model without requiring a local model installation.
- Streamlit provides the user interface for audio upload and text display.
- The system can be extended by connecting the transcribed text to a chatbot or translation module.